

Single Cycle Processor

Course Project — CE222

Your Name

May 4, 2026

Outline

- 1 Introduction
- 2 Architecture
- 3 Modules
- 4 Verification
- 5 Results
- 6 Conclusion

Project Objectives

- Implement a single-cycle RISC-like processor in Verilog
- Verify functionality with testbenches and waveform inspection
- Demonstrate datapath, control, and memory integration
- Present simulation results and example executions

Top-level Block Diagram

Missing block_diagram.png
Please add exported diagram here

(Insert

annotated datapath: IF, ID, EX, MEM, WB)

Datapath Overview

- Instruction Fetch: Program Counter, Instruction Memory
- Instruction Decode: Register File, Immediate Generator, Control Unit
- Execute: ALU, ALU Control, MUXes
- Memory Access: Data Memory (load/store)
- Write Back: Register update from ALU or Memory

Key Verilog Modules

- **Instruction Memory:** `instruction_memory.v`
- **Register File:** `register_file.v`
- **Program Counter:** `pc.v`
- **ALU:** `alu.v`, `alu_control.v`
- **Control Unit:** `control.v`
- **Data Memory:** `data_memory.v`

Register File

- 32 registers, synchronous write, asynchronous read
- Ports: read1, read2, write, writeData, writeEnable
- Testbench: `register_file_tb.v` (use to validate read/write semantics)

ALU and ALU Control

- ALU supports add, sub, and, or, slt, shifts, etc.
- `alu_control.v` maps opcode/funct fields to ALU ops
- Unit-tested with `test_alu` in repo

Control Unit

- Decodes opcode to generate control signals (RegWrite, MemRead, MemWrite, ALUSrc, MemToReg)
- Single-cycle control: all signals valid in one clock cycle per instruction
- File: `control.v`

Testbenches and Simulation

- Use iverilog/vvp to simulate and gtkwave to view waveforms
- Primary testbench: `single_cycle_processor_tb.v`
- Additional benches in repo: `instruction_fetch_tb.v`, `register_file_tb.v`, etc.

Suggested Simulation Commands

```
# compile
iverilog -o simv *.v *_tb.v
# run
vvp simv
# view waveforms (if .vcd produced)
gtkwave single_cycle_processor.vcd
```

Example Waveforms

Missing waveform_example.png
Export VCD screenshots or PNG here

Show

instruction execution, PC updates, register writes, memory accesses.

Functional Results

- Correct execution of sample instruction sequences verified via VCD
- Register read/write and memory load/store validated in testbenches
- Timing: single-cycle latency determined by longest combinational path (ALU + memory)

Performance / Observations

- Pros: Simple control, easy to reason about, minimal pipeline hazards
- Cons: Single-cycle restricts clock frequency due to long-critical paths
- Potential improvements: pipelining, forwarding, hazard detection

Conclusion

- Implemented a working single-cycle processor in Verilog
- Verified with testbenches and waveform inspection
- Next steps: add pipeline stages and hazard logic for performance

Appendix: Files of Interest

- `single_cycle_processor.v` – top-level (Single_Cycle_Processor folder)
- `single_cycle_processor_tb.v` – top-level testbench
- `instruction_memory.v`, `data_memory.v` – memories
- `register_file.v`, `alu.v`, `control.v`, `pc.v` – core modules

How to Build the Presentation

- Compile: `pdflatex presentation.tex` (run twice for TOC)
- Include exported PNGs for block diagram and waveforms in the same folder
- Recommended: use Google Slides / PowerPoint export if required